

Ala-Alab

Human-AI Methodology for Community Memory Systems

Giving communities the freedom to remember.

Project	Ala-Alab — SparkFest 2026	Competition	GDG PUP SparkFest 2026
Builder	CS Student, PUP Valenzuela	Sprint	5-Day Solo Build (Jun 28 – Jul 2)
Version	Methodology v1.0	Date	June 29, 2026

Overview

Ala-Alab is a living AI-powered system for barangay community memory. It captures oral testimony, field observations, narrative reports, and official records into a single structured document — a context file that any AI agent can read and act on immediately, without re-establishing context from scratch. The methodology below defines how the system is built, maintained, and used across the full sprint.

Sprint Timeline

Days are split by sleep, not by midnight. Each day begins when the builder wakes.

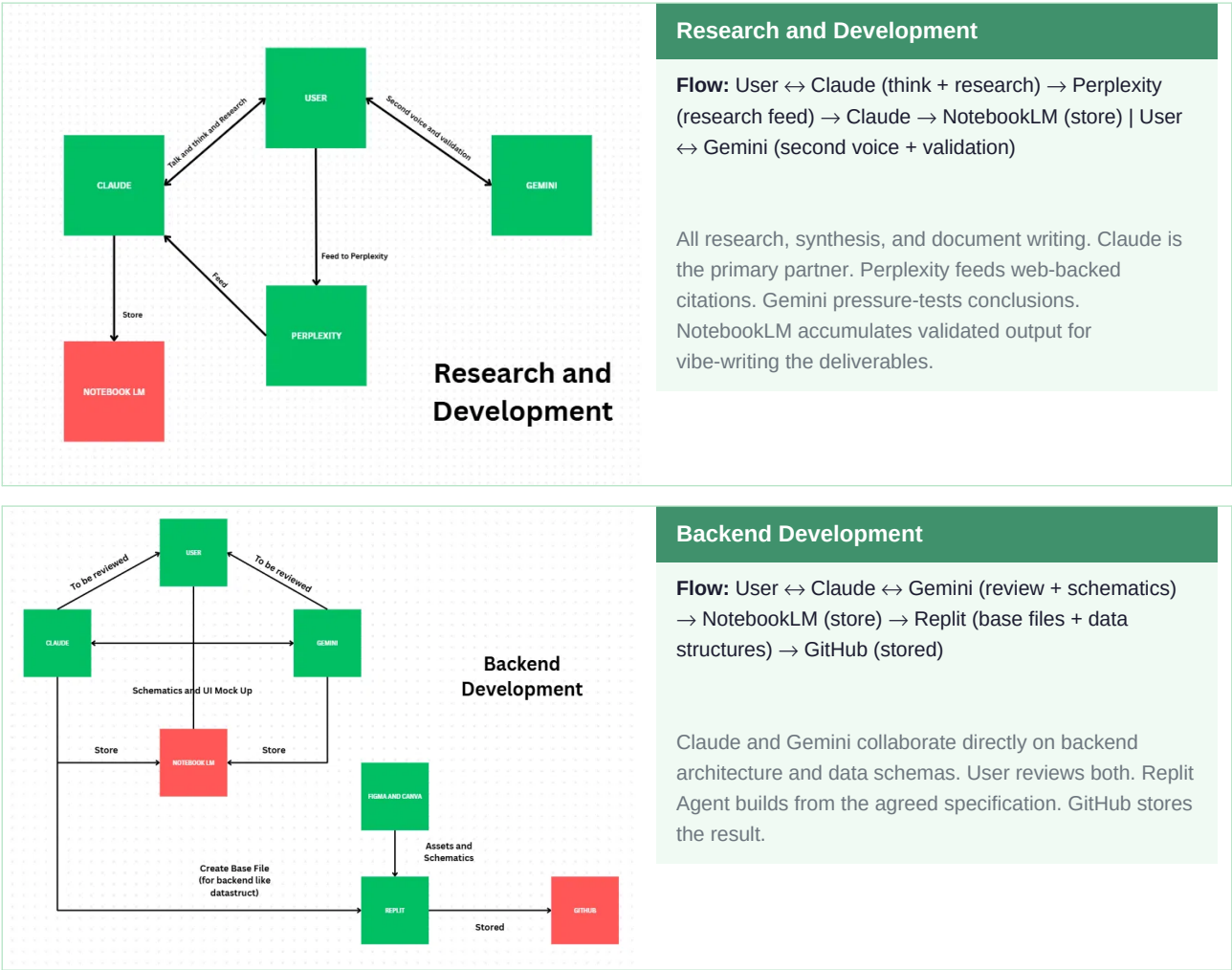
Day	Date	Focus	Key Output
1	Jun 28 (Complete)	Repo setup. Account creation. Infrastructure and workflow planning.	GitHub repo live. Workflow diagrams drafted.
2	Jun 29 (Today)	Agentic interaction design. instructions.md. Schema draft. Project brief. Pitch document.	instructions.md One-page brief draft
3	Jun 30	Full build. UI assets. Backend + frontend. Multi-session testing. Google Auth + Vercel deploy.	Working MVP. Demo footage captured.
4	Jul 1	Refine and test. More datasets. Simulate with AI + user. Finalize docs. Video plan.	Polished build. Video script ready.
5	Jul 2	Final simulation. Repo cleanup. Document finalization. Video record and submit.	Submitted. Commit history complete.

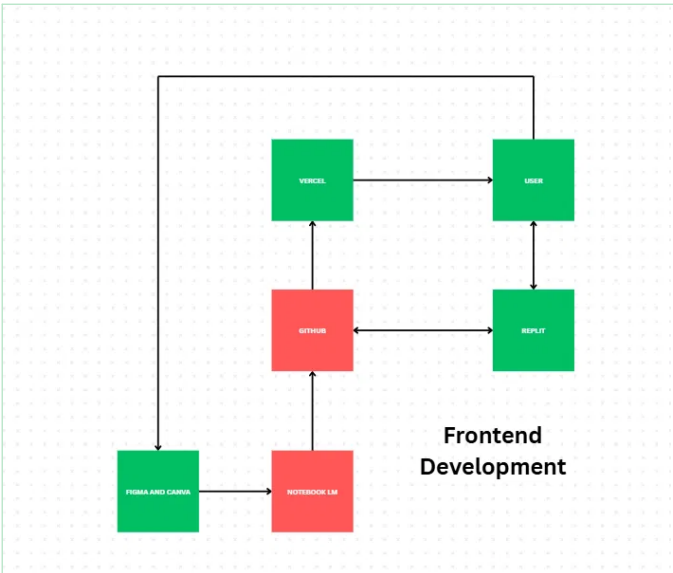
Agent Roles

Agent	Role	Primary Workflow
Claude	Primary thinking partner. Logic, document maintenance, structured writing, erratum updates, script drafting.	R&D, Writing, Backend
Gemini Flash	Second voice and validation. Multimodal intake (photo + text). Conversational interface for barangay workers.	R&D, Backend, App
Replit Agent	App builder. Scaffolds and builds the application. User directs, agent codes.	Backend, Frontend
NotebookLM	Accumulator and synthesizer. Holds all research, Dredge content, and outputs across all four workflows.	All workflows

Perplexity	Research feed. Citation-backed web queries fed into Claude for R&D tasks.	R&D only
GitHub Copilot	Repo-level code fixes during testing. Works inside VS Code on the live codebase.	Testing
VS Code	Code review and surgical edits on Replit Agent output.	Frontend, Testing
Figma + Canva	UI mockups and design assets. User-created, fed into Replit as build reference.	Frontend

The Four Workflows

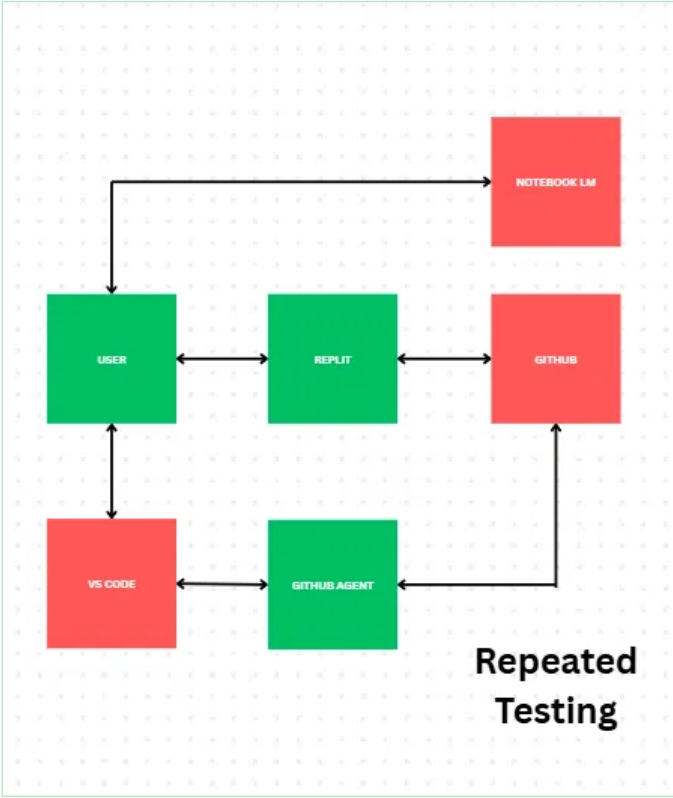




Frontend Development

Flow: User → Canva (UI mockups) → Figma (schematics) → Replit (assets + build reference) ↔ GitHub ↔ Vercel (deploy) → User

User designs assets in Canva, refines in Figma, feeds into Replit as the build reference. Replit and GitHub stay in sync. Vercel deploys the live product. Claude and Gemini inform design through review.



Repeated Testing

Flow: User ↔ Replit ↔ GitHub → NotebookLM | User ↔ VS Code ↔ GitHub Copilot ↔ GitHub

Two parallel loops. Replit for live session testing, VS Code + Copilot for repo-level surgical fixes. NotebookLM accumulates test findings. GitHub is the single source of truth throughout.